



•OPERATING SYSTEM

•INSY 8223

Faculty of IT, GROUP D&F

Dr Eric NIZEYIMANA and Mr. Archimede IRAKOZE

eric.nizeyimana@auca.ac.rw | +250 783 308 544

Lesson 8. Device and I/O Management

Introduction to Device & I/O Management

OS controls and coordinates all I/O operations

Ensures devices are used efficiently and safely

Hides hardware complexity from user programs

Provides uniform interface to different devices

Introduction

Device and I/O management in an operating system (OS) refers to how the OS controls the flow of data between the computer and its external devices, also known as peripheral devices.

This includes managing the access, transfer, and control of data between the computer's I/O devices, such as keyboards, mice, and printers, and the rest of the system.

Types of I/O Devices

1. Block Devices

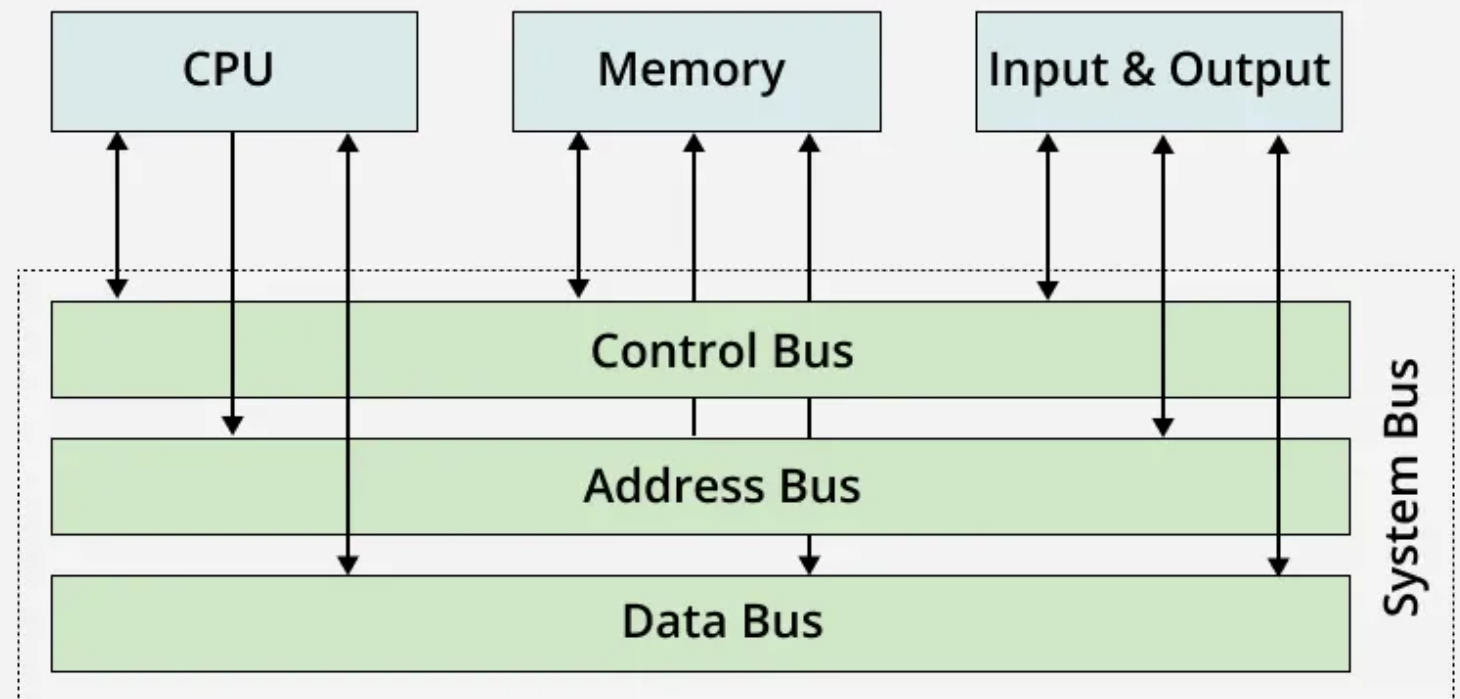
- Store data in fixed-size blocks
- Random access possible
- Examples: HDD, SSD, USB storage

2. Character Devices

- Stream of characters
- No random access
- Examples: keyboard, mouse, serial ports

Device Controller

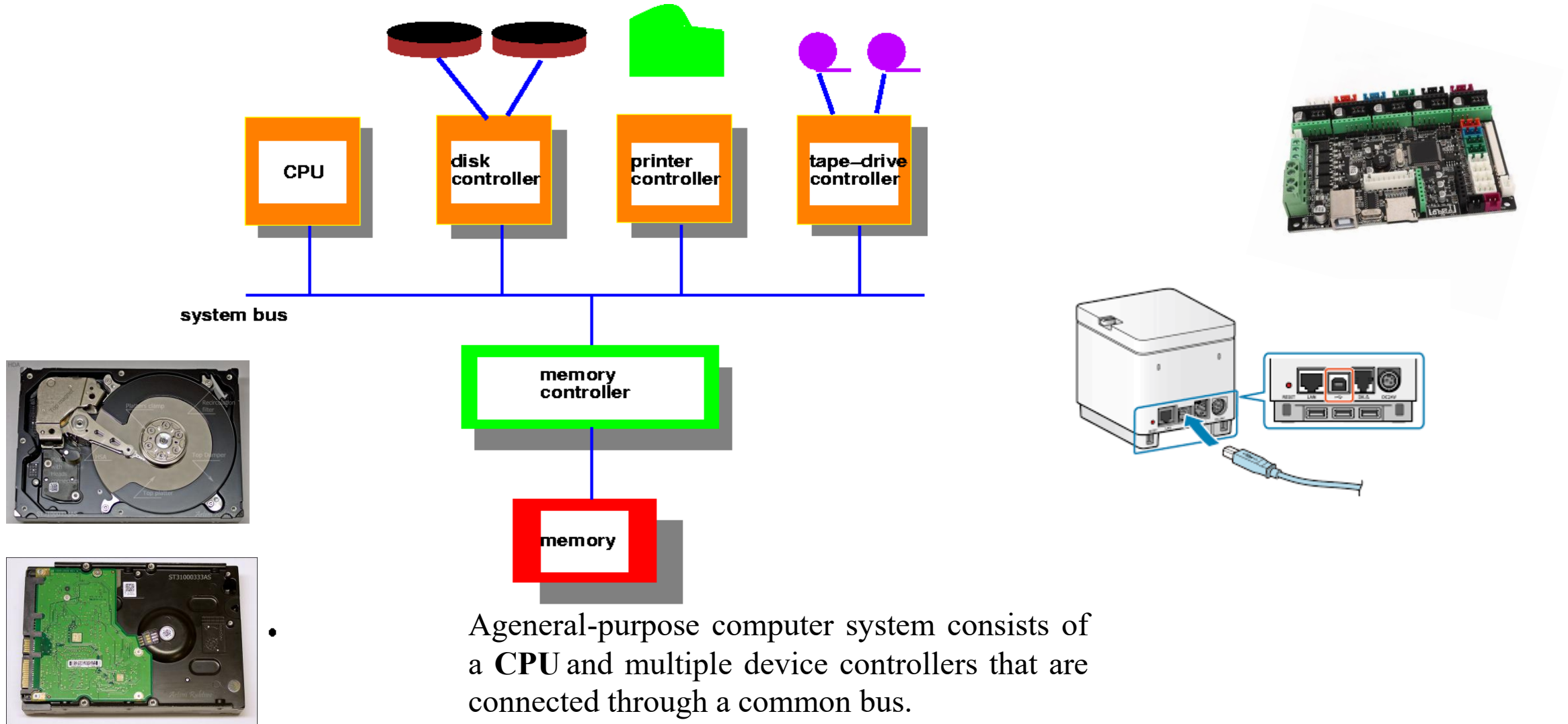
- Small processor inside the device
- Interfaces between device and CPU
- Converts signals
- Examples:
 - Disk controller
 - USB controller
 - Network interface card (NIC)



The OS acts as a mediator between the devices and the running programs, ensuring efficient and orderly access to these resources. Here are some ways the OS manages I/O:

- **Device drivers:** The OS uses device drivers to communicate with hardware.
- **Interrupts:** The OS handles tasks with interrupts to stay efficient.
- **Buffering:** The OS uses buffering to manage data speed differences.

A Modern Computer System



Computer System Organization (I/O Structure)

I/O devices and the CPU execute concurrently.

Each device controller is in charge of a particular device type

- Each device controller has a **local buffer**. I/O is from the device to local buffer of controller

CPU moves data from/to main memory to/from the local buffers

Device controller **interrupts** CPU on completion of I/O

I/O Communication Techniques

1. Programmed I/O

- CPU polls the device repeatedly
- Simple but inefficient

2. Interrupt-Driven I/O

- Device signals CPU via interrupts
- CPU handles I/O only when needed

3. Direct Memory Access (DMA)

- Controller transfers data directly to memory
- Frees CPU from transfer tasks
- Best for high-speed devices

Interrupts

Interrupt: is a signal emitted by hardware or software when a process or an event needs immediate attention. It alerts the processor to a high-priority process requiring interruption of the current working process.

Interrupt transfers control to the interrupt service routine

- **Interrupt Service Routine:** Segments of code that determine action to be taken for each type of interrupt.
- Interrupt vector contains the address of service routines.

OS preserves the state of the CPU

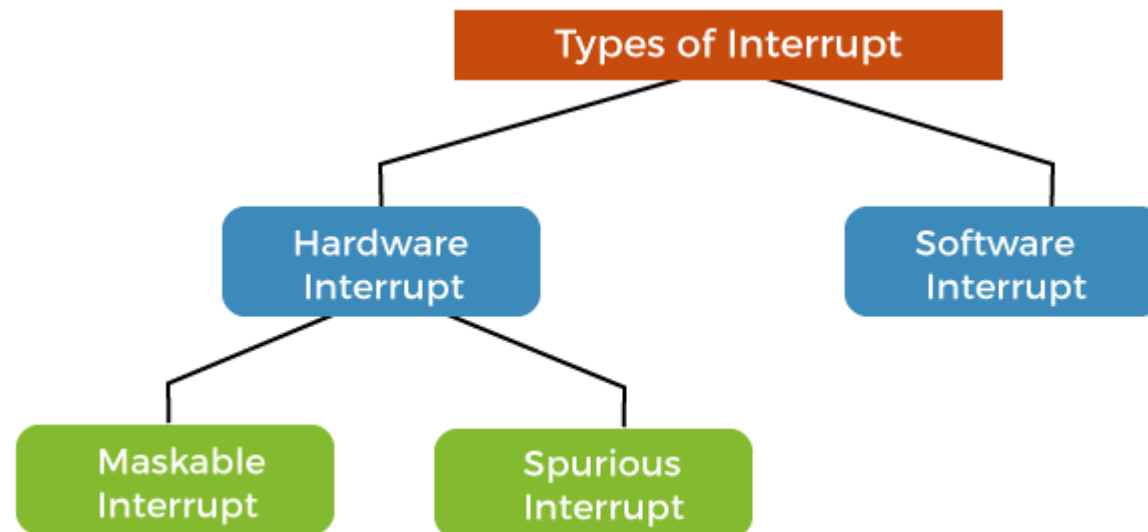
- stores registers and the program counter (address of interrupted instruction).
- After the interrupt is serviced, the saved return address is loaded into the program counter, and the interrupted computation resumes as though the interrupt had not occurred.

Trap

- software generated interrupt caused either by an error or a user request.

TYPES OF INTERRUPT

Interrupt signals may be issued in response to hardware or software events. These are classified as *hardware interrupts* or *software interrupts*, respectively.



Hardware Interrupts

A **hardware** interrupt is a condition related to the state of the hardware that may be signaled by an external hardware device. For example, pressing a keyboard key or moving a mouse triggers hardware interrupts that cause the processor to read the keystroke or mouse position.

Hardware interrupts can arrive asynchronously for the processor clock and at any time during instruction execution. Consequently, all hardware interrupt signals are conditioned by synchronizing them to the processor clock and act only at instruction execution boundaries.

Hardware Interrupts (cont.): (Maskable vs Spurious interrupts)

1. Maskable Interrupts: Processors typically have an internal interrupt mask register which allows selective enabling and disabling of hardware interrupts. Each interrupt signal is associated with a bit in the mask register; on some systems, the interrupt is enabled when the bit is set and disabled when the bit is clear, while on others, a set bit disables the interrupt.

- To **mask** an interrupt is to disable it, while to **unmask** an interrupt is to enable it.

2. Spurious Interrupts: A spurious interrupt is a hardware interrupt for which no source can be found. The term phantom interrupt or ghost interrupt may also use to describe this phenomenon. Such interrupts may be difficult to identify when a system misbehaves.

A spurious interrupt may result in system deadlock or other undefined operation if the Interrupt Service Routine (ISR) doesn't account for the possibility of such an interrupt occurring.

To solve the issue, ISR has to check all interrupt sources for activity and take no action if none of the sources is interrupting.

Software Interrupts

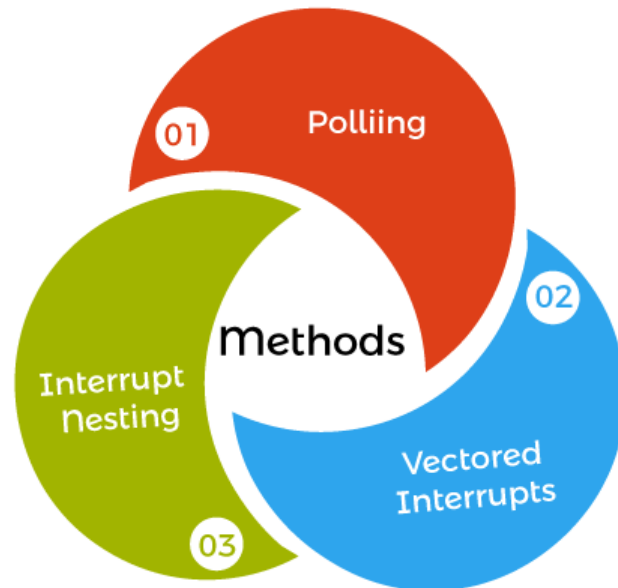
The processor requests a software interrupt upon executing particular instructions or when certain conditions are met. Every software interrupt signal is associated with a particular interrupt handler.

A software interrupt may be intentionally caused by executing a special instruction that invokes an interrupt when executed by design. Such instructions function similarly to subroutine calls and are used for various purposes, such as requesting operating system services and interacting with device drivers.

Software interrupts may also be unexpectedly triggered by program execution errors. These interrupts are typically called ***traps*** or ***exceptions***.

Handling Multiple Devices

When more than one device raises an interrupt request signal, additional information is needed to decide which device to consider first. The following methods are used to decide which device to select first.



Interrupt Handling

Types of interrupt

- **Polling** : the first device encountered with the IRQ bit set is to be serviced first, and appropriate ISR is called to service the same. It is easy to implement, but a lot of time is wasted by interrogating the IRQ bit of all devices.
- A **polled interrupt** is a specific type of I/O **interrupt** that notifies the part of the computer containing the I/O interface that a **device is ready to be read** or otherwise handled but **does not indicate which device**.
- **Vectored interrupt system** : In vectored interrupts, a device requesting an interrupt identifies itself directly by sending a special code to the processor over the bus. This enables the processor to identify the device that generated the interrupt. The special code can be the starting address of the ISR or where the ISR is located in memory and is called the **interrupt** .
- In a nutshell (in few words), vectored interrupt is alternative to polling since this requires that the interrupt handler poll or send a signal to each device in turn in order to find out which device sent interrupt request.
- **Interrupt Nesting** : In this method, the I/O device is organized in a priority structure. Therefore, an interrupt request from a higher priority device is recognized, whereas a lower priority device is not. The processor accepts interrupts only from devices/processes having priority more than it.

I/O Interrupt Handling

Device triggers interrupt

CPU pauses the current task

OS interrupt handler executes

ISR communicates with driver

System resumes program execution

I/O Interrupts

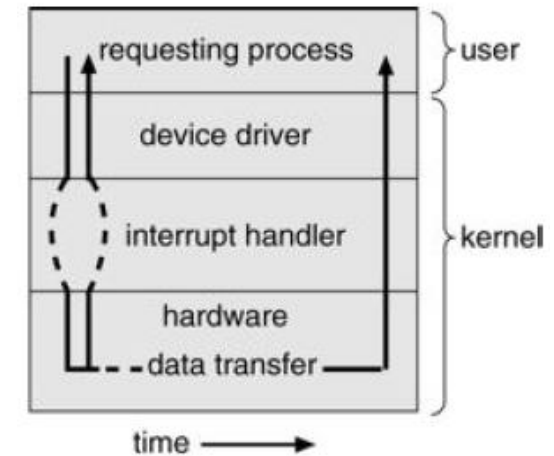
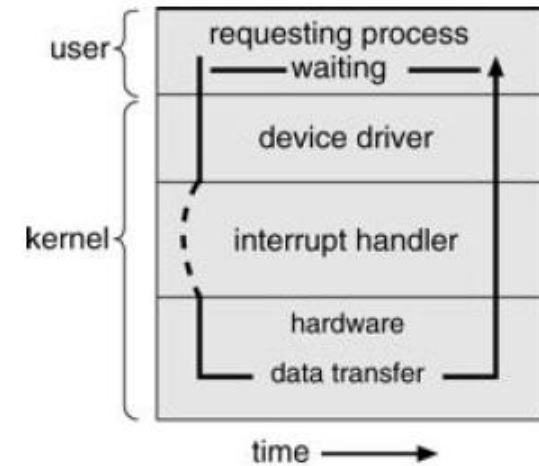
Synchronous I/O

- *wait* instruction idles CPU until next interrupt
- no simultaneous I/O processing, at most one outstanding I/O request at a time.

Asynchronous I/O

After I/O is initiated, control returns to user program without waiting for I/O completion.

- System call
- **Device Status table** - holds type, address and state for each device
- OS indexes into I/O device table to determine device status and modify table entry to include interrupt.



Synchronous and asynchronous I/O

Synchronous I/O is *blocking*, meaning a program waits for an I/O operation to complete before moving on.

Asynchronous I/O is *non-blocking*; it sends an I/O request and continues processing other tasks until the operation is complete, which improves performance by keeping the program responsive.

Device Drivers

- Software that manages I/O devices
- Translates OS instructions to device commands
- Types:
 - Kernel-mode drivers
 - User-mode drivers
 - Plug-and-play drivers

Communication to I/O Devices

The CPU must have a way to pass information to and from an I/O device. There are three approaches available to communicate with the CPU and Device.

- Special Instruction I/O (I/O Interrupts)
- Memory-mapped I/O
- Direct memory access (DMA)

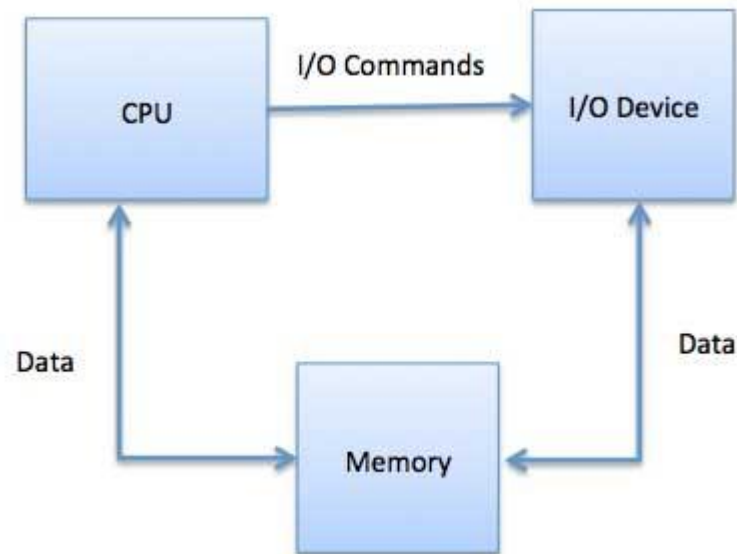
SPECIAL INSTRUCTION I/O (IO Interrupts)

This uses CPU instructions that are specifically made for controlling I/O devices. These instructions typically allow data to be sent to an I/O device or read from an I/O device.

Memory-mapped I/O

When using memory-mapped I/O, the same address space is shared by memory and I/O devices.

The device is connected directly to certain main memory locations so that I/O device can transfer block of data to/from memory without going through CPU.



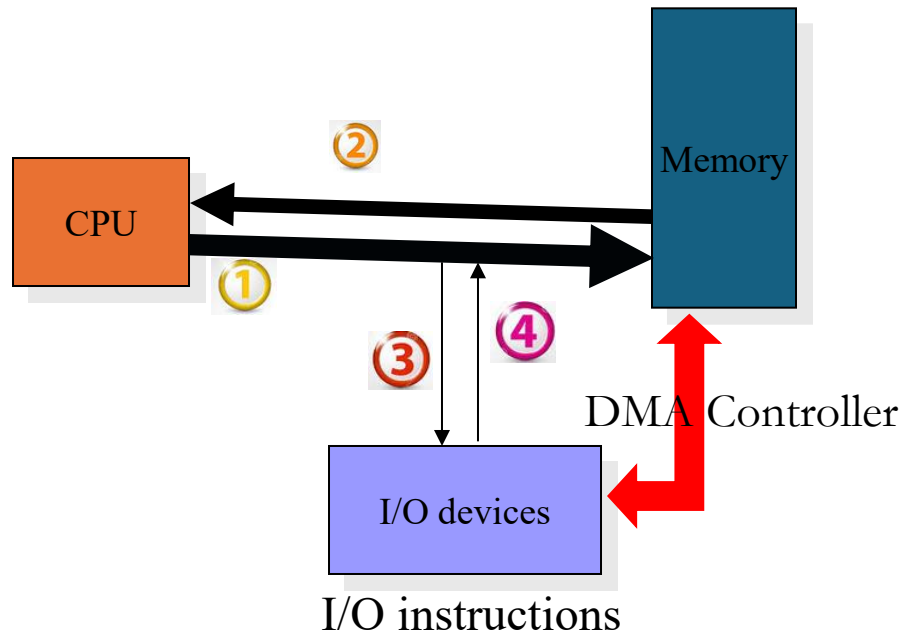
Memory-mapped I/O (cont..)

- While using memory mapped IO, OS allocates buffer in memory and informs I/O device to use that buffer to send data to the CPU.
- I/O device operates asynchronously with CPU, interrupts CPU when finished.
- The advantage to this method is that every instruction which can access memory can be used to manipulate an I/O device.
- Memory mapped IO is used for most high-speed I/O devices like disks, communication interfaces.

Direct Memory Access (DMA)

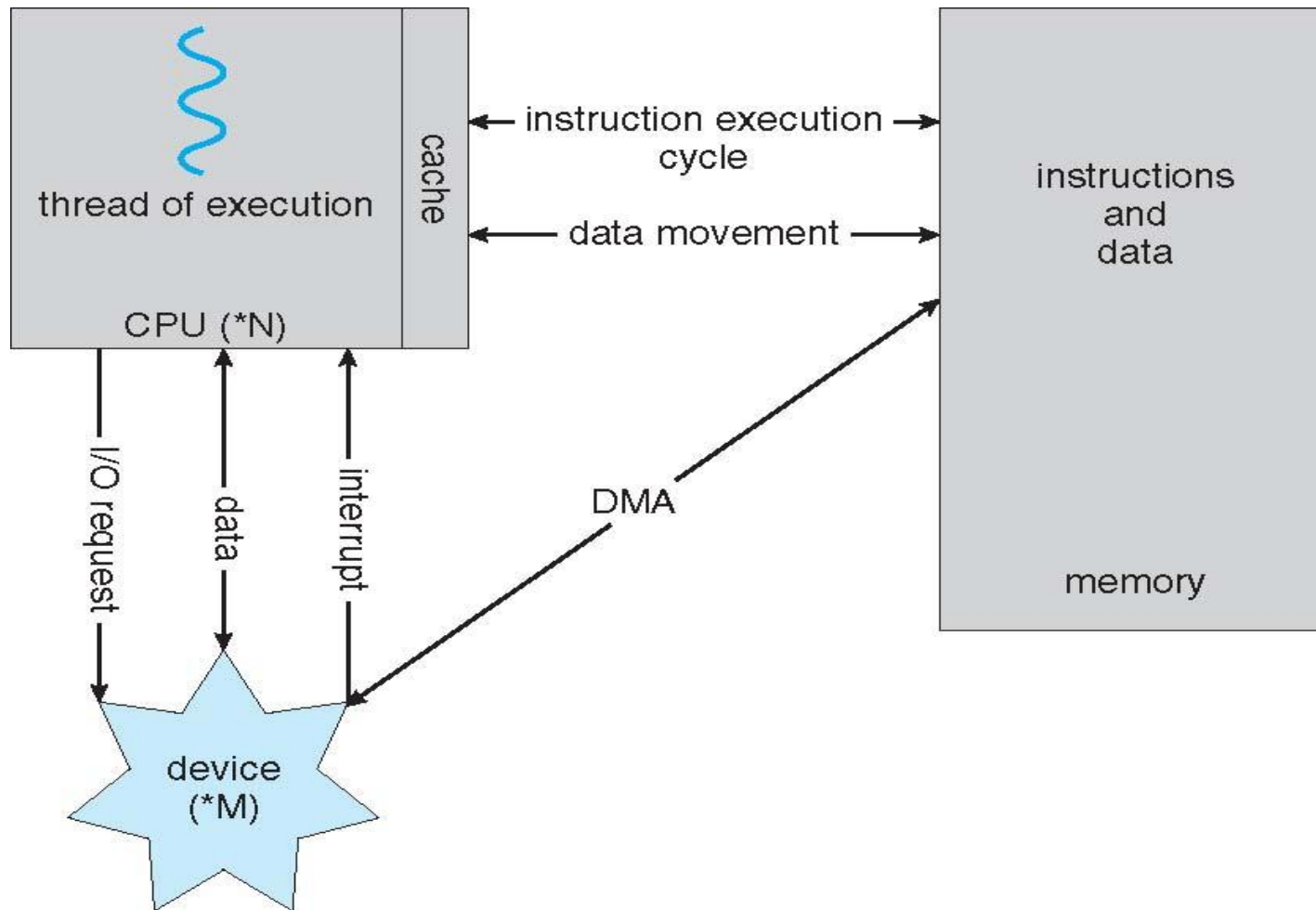
- Slow devices like keyboards will generate an interrupt to the main CPU after each byte is transferred. If a fast device such as a disk generated an interrupt for each byte, the operating system would spend most of its time handling these interrupts. So a typical computer uses direct memory access (DMA) hardware to reduce this overhead.
- Direct Memory Access (DMA) means CPU grants I/O module authority to read from or write to memory without involvement. DMA module itself controls exchange of data between main memory and the I/O device. CPU is only involved at the beginning and end of the transfer and interrupted only after entire block has been transferred.

Direct Memory Access (DMA)



- **CPU initiates:** The CPU sets up the DMA controller by providing the source and destination addresses, as well as the amount of data to transfer.
- **DMA controller takes over:** The DMA controller then takes control of the system bus to perform the data transfer directly between the peripheral and memory.
- **CPU is freed:** While the DMA controller is working, the CPU is free to execute other instructions.
- **Completion signal:** Once the transfer is complete, the DMA controller sends an interrupt to the CPU to signal that the operation is finished.

How a Modern Computer Works



A von Neumann architecture

Comparison of Communication Methods to I/O Devices in Operating Systems

Feature	Special Instruction I/O	Memory-Mapped I/O	Direct Memory Access (DMA)
Communication Mechanism	Uses specific CPU instructions for I/O operations.	Maps I/O device registers to memory addresses.	Allows devices to transfer data directly to/from memory without CPU involvement.
CPU Involvement	High; CPU manages all data transfers.	Moderate; CPU reads/writes data using memory operations.	Low; CPU is only involved in setting up the transfer.
Addressing	Separate I/O address space.	Unified memory and I/O address space.	Operates in memory address space.
Instruction Complexity	Requires dedicated I/O instructions.	Uses regular memory read/write instructions.	Requires DMA controller setup.
Speed	Slower due to direct CPU involvement in I/O.	Faster than Special Instruction I/O due to direct memory mapping.	Fastest; bypasses CPU for bulk data transfers.
Implementation Complexity	Simpler hardware but limited flexibility.	More flexible but requires careful memory allocation.	Complex hardware (DMA controller) is required.
Usage Scenarios	Suitable for low-speed devices or simple systems.	Common in modern systems for ease of programming.	Ideal for high-speed devices requiring large data transfers.

I/O Scheduling

- Determines the order in which I/O requests are handled
- Goals: reduce latency, maximize throughput
- Common algorithms:
 - FCFS
 - SSTF

Buffering

- **Why buffering?**
 - Smooth speed mismatch between devices
 - Support different transfer sizes
 - Increase I/O efficiency
- **Types:**
 - Single buffer
 - Double buffer
 - Circular buffer

Caching in I/O

- Store frequently accessed data in fast memory
- Reduces I/O wait time
- Examples:
 - Disk cache
 - File system cache
 - Browser cache

I/O Protection & Security

- I/O ports protected by OS
- Direct hardware access restricted
- Secure device communication
- Prevent malicious device drivers

Performance Optimization Techniques

- DMA
- I/O scheduling
- Caching
- Buffering
- Multithreaded I/O
- Asynchronous I/O

Lesson 9. Practical in both Windows and Linux

Thank you



For I know the thoughts that I think toward you, saith the LORD, thoughts of peace, and not of evil, to give you an expected end. Jeremy 29:11